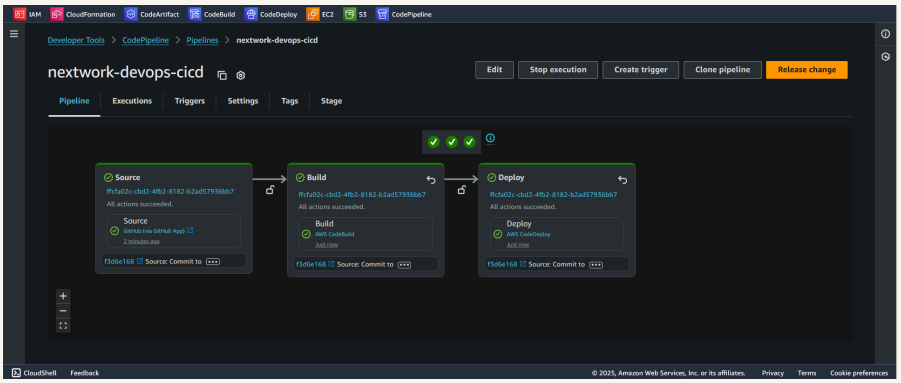
nextwork.org

Build a CI/CD Pipeline with AWS

Muhammad Sadique
https://www.linkedin.com/in/muhammad-sadique-



Muhammad Sadique
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate automation of CI/CD pipeline components through AWS CodePipeline. I am doing this project to learn CI/CD pipeline concepts.

Key tools and concepts

Services I used were CodeBuild, CodeDeploy, and CodePipeline. Key concepts that i learnt include: 1. Automation of CI/CD workflow through AWS CodePipeline 2. Rollback to last commit if new release have problem

Project reflection

This project took me approximately 6 hours. The most challenging part was setting up pipeline and IAM Roles and permissions.

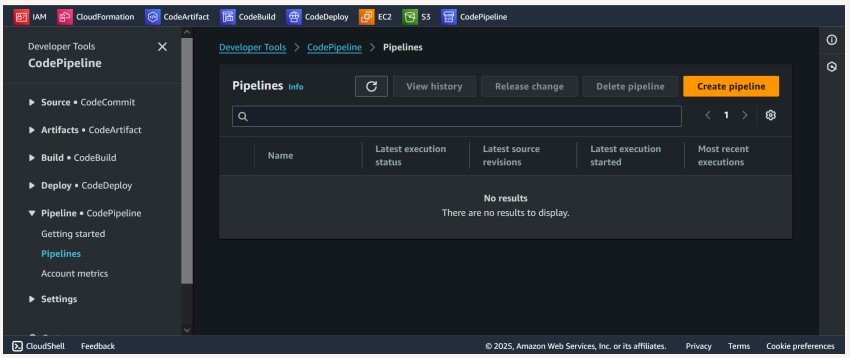
Muhammad Sadique
NextWork Student

Starting a CI/CD Pipeline

AWS CodePipeline is a CI/CD tool where you can create workflows to automatically move your code changes through the build and deployment stage. A new push to my GitHub repository automatically triggers a build in CodeBuild (continuous integration), and a then a deployment in CodeDeploy (continuous deployment)!

CodePipeline offers different execution modes based on concurrency. I chose "superseded mode" to allow the latest execution to run and cancel any ongoing execution. This will ensure that latest code changes are always processed, ideal for CI/CD pipelines where you always want the most recent version deployed.

A service role gets created automatically during setup to delegate AWS CodePipeline necessary permissions required to access other AWS resource like S3 bucket for storing artifacts or CodeBuild for building your code.

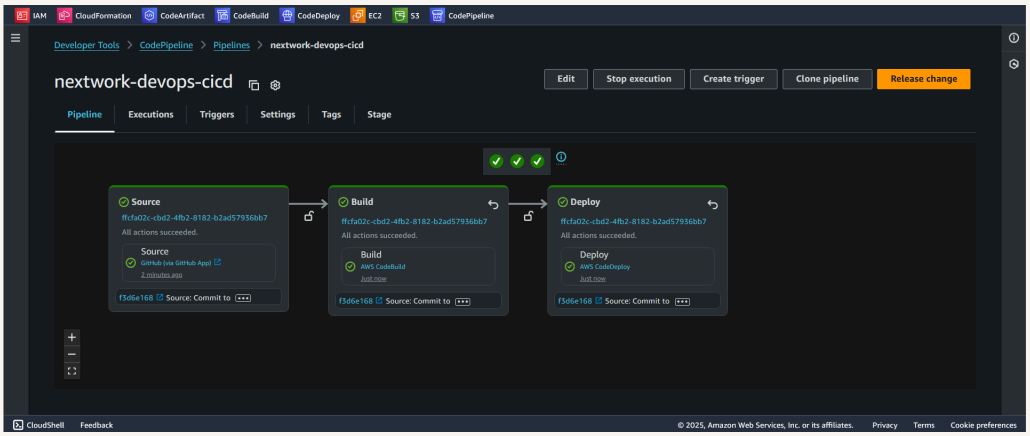


Muhammad Sadique
NextWork Student

CI/CD Stages

The three stages I've set up in my CI/CD pipeline are: 1. Source 2. Build 3. Deploy

CodePipeline organizes the three stages into source, build and deploy. In each stage you can see more details by clicking stage ID.

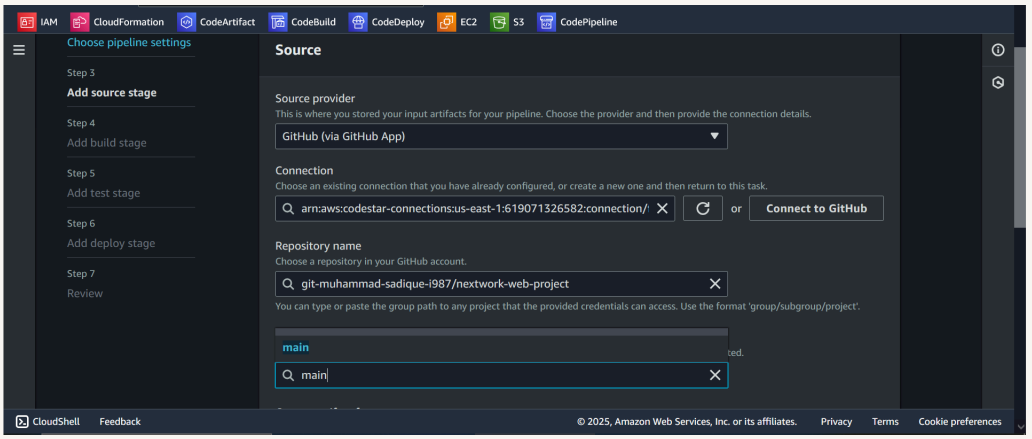


Muhammad Sadique
NextWork Student

Source Stage

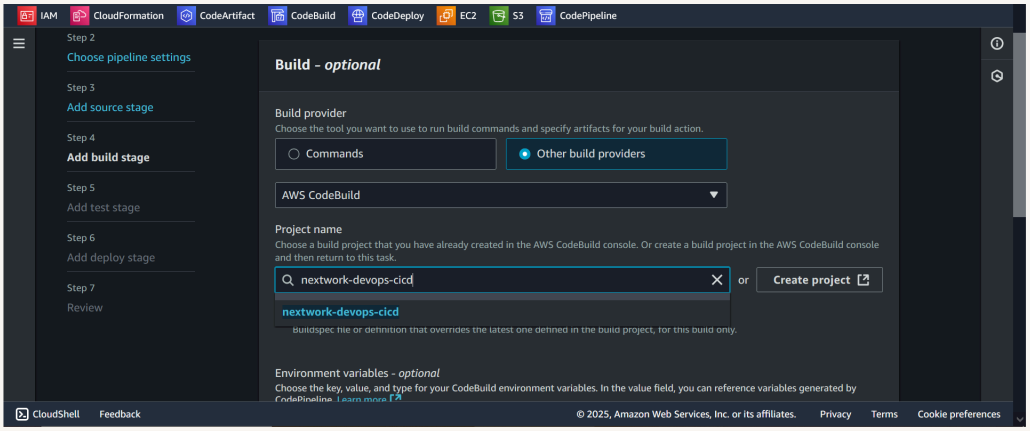
In the Source stage, the default branch tells CodePipeline to monitor this branch for changes and trigger the pipeline whenever there's a commit to it.

The source stage is also where we enable webhook events, which are digital notifications from GitHub to CodePipeline in response to specific events such as code push to the main branch of our GitHub repository.



Build Stage

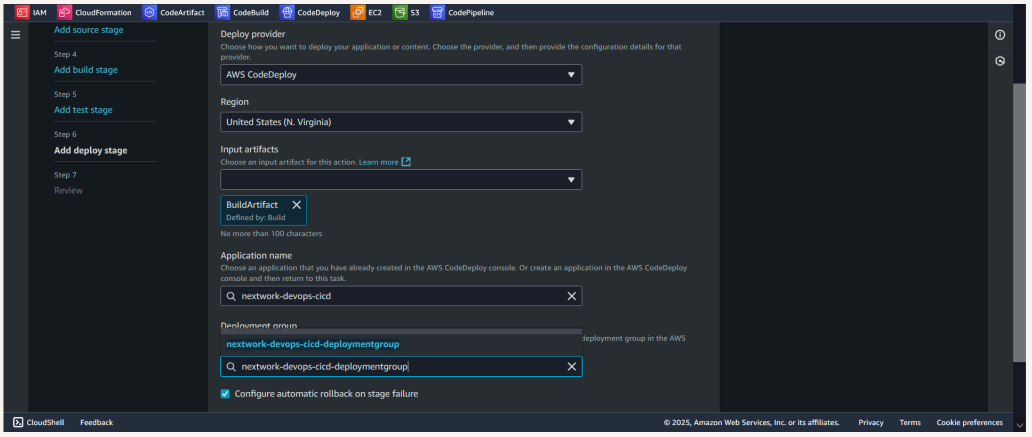
The Build stage sets up our code into a deploy-able package. I configured previously created build project in AWS CodeBuild at this stage. The input artifact for the build stage is output from our previous stage i.e. source stage.



Muhammad Sadique
NextWork Student

Deploy Stage

The Deploy stage is where AWS CodePipline take the application artifacts and deploy them to the target environment, which in our case is EC2 instance. To achieve this, i configured AWS CodePipeline Deploy stage. From drop down, I chose AWS CodeDeploy as "Deploy provider". Afterwards, I selected application and deployment group already created in AWS CodDeploy.

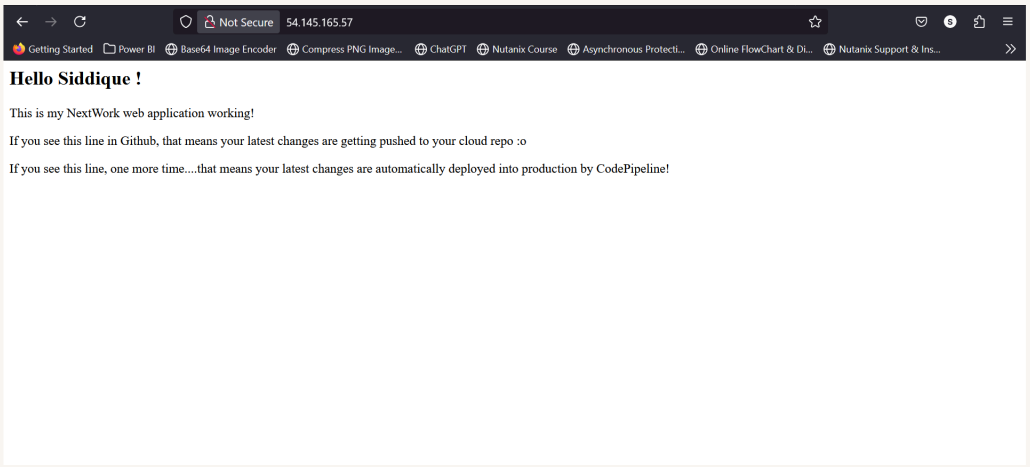


Success!

Since my CI/CD pipeline gets triggered by code change, i tested my pipeline by appending additional lines inside my code. All three stages went find and i could see the changes in production.

The moment I pushed the code change, it reflected in my GitHub repository as well. GitHub notified AWS CodePipeline regarding code change through webhook, hence the pipeline workflow started. The commit message under each stage reflects the code changes since last commit.

Once my pipeline executed successfully, I checked my web application. I saw additional lines i added to my code there which proves that pipeline was working as intended.


Not Secure
54.145.165.57
Getting Started
Power BI
Base64 Image Encoder
Compress PNG Image...
ChatGPT
Nutanix Course
Asynchronous Protecti...
Online FlowChart & Di...
Nutanix Support & Ins...
Hello Siddique !
This is my NextWork web application working!
If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o
If you see this line, one more time....that means your latest changes are automatically deployed into production by CodePipeline!

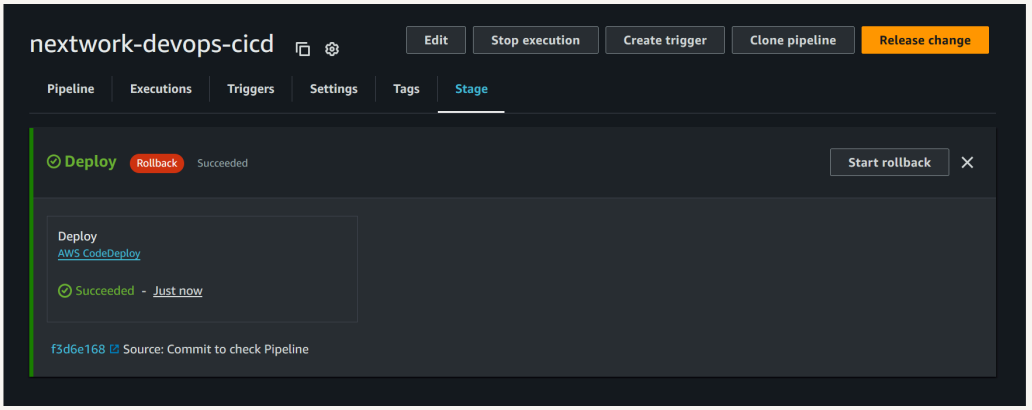
Testing the Pipeline

In a project extension, I initiated a rollback on deploy stage. Automatic rollback is important, if the latest changes caused problems! In this case, Ionly rolled back the deploy stage. This means my source code and build stay the same, but the application version deployed to the server is reverted back to how it was before my latest commit.

During the rollback, the source and build stage are found unaffected by the rollback because i initiated the rollback onto deploy stage. I could verify this by comparing commit ID's on each stage. The Source and the Build stage had similar commit ID whereas Deploy stage had different commit ID.

After rollback, the live web app reverted to last commit.

Muhammad Sadique
NextWork Student



The place to learn & showcase your skills

Check out nextwork.org for more projects